ReacherX Benchmark v2

Proposal for approval · 10 November 2026

Purpose	Measure, with evidence, how well ReacherX finds the right people compared with other tools, whether it improves over time, and whether a cheaper AI model can do the same job.
Compared with	Exa, Apollo, Lusha, Treg, Clay, context.dev, Firecrawl and Crustdata; 20 prompts, reported by supported task.
How results are checked	Every result is graded by a panel of AI judges that never know which tool returned it, and the judges are checked against human reviewers.
Cost	Base priced-call allowance: $24–43 for stage 1 and $28–114 for stage 2, about $52–157. Quota usage is tracked separately; pilot measurements and available credits determine cash required.
Effect on the product	None for customers. Runs use a private development copy of ReacherX, with benchmark-only learning and development-only concurrency controls.

1. Summary

ReacherX finds people to reach out to by searching X and LinkedIn in regular rounds, called cycles, and by learning from what it finds. This proposal sets up a benchmark, built into the ReacherX code in TypeScript and rerunnable at any time, that tests these strengths with data. Stage 1 builds the benchmark, proves that it works and compares ReacherX with the other tools; stage 2 runs ReacherX for longer to see whether it learns. Each step is paid for only after the previous one has shown its real cost and results, and every result is published, including losses. The proposal builds on the original brief, the project lead's review of 10 November 2026 and the vendor credits arranged since.

Key terms

Prompt	A plain-English description of the people or companies to find, for example "senior product designers in Germany who mentor junior designers".
Cycle	One round of ReacherX's work: it searches X and LinkedIn, then scores each person it finds. In production, cycles run every hour after a faster first hour.
Workspace	A separate ReacherX setup with its own prompt and history. Each test runs in its own workspace.
Learning	ReacherX keeps lessons from earlier cycles and uses them to search and score better in later ones.
Verified match	A person or company meeting every must-have requirement of the prompt, confirmed with evidence.

AI judge	An AI model that grades each result against the prompt, without knowing which tool returned it.

2. What will be compared

2.1 The prompts

Twenty approved prompts in four groups, listed in Appendix A: people with static filters (7), people with signals (5), companies with static filters (5), and companies with signals (3). Static prompts test match quality; the eight signal prompts also probe freshness. Untimed signals carry evidence dates without an added age cutoff.

Before live runs, freeze the criteria, definitions and reference date, and confirm qualifying examples exist. Report all four groups separately. ReacherX supports organization prospects; keep company prompts in scope and verify their result path in the smoke test. Grade the company itself, not an employee. Document unsupported capabilities before comparison; an empty result alone does not establish lack of support.

2.2 The tools

Tool	How it is used
ReacherX	Production search and models, Starter concurrency, one development workspace at a time
Exa	People or company search, matching the prompt type
Apollo	Through Treg. Apollo hides surnames, so each person needs a paid lookup before judging
Lusha	Through Treg
Treg	Its own people search. Treg sometimes passes a search to another provider; this is recorded
Crustdata	Through Treg: indexed people and company search. Log the endpoint and mode; do not combine it with real-time search
Clay	Its own API, on the existing account, at no cost
context.dev	People search, on the project lead’s account (50,000 credits granted)
Firecrawl	Its search

For filter-based tools, one shared, logged step turns each prompt into native filters without dropping requirements. Unsupported capabilities are recorded. Two tools from the original brief are left out: ZoomInfo, which cannot be reached through Treg and has no public API pricing, and Firecrawl Alexandria, a gateway to providers already tested, such as Apollo.

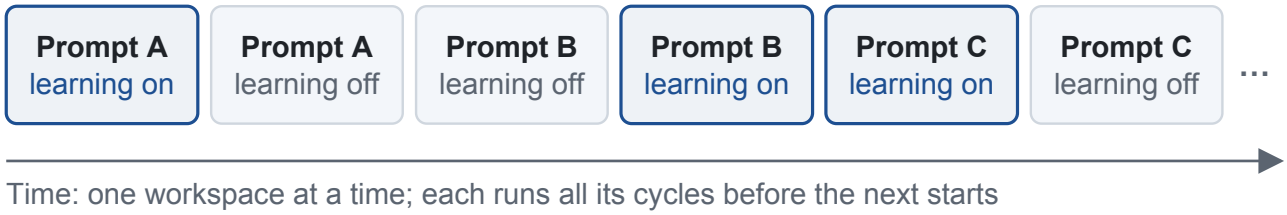
3. How the benchmark works

3.1 Two kinds of test

	Quick check	Full run
What it does	Re-scores a fixed set of about 300 people whose right answers are already known	Runs ReacherX for real over several cycles, alongside the other tools
Used for	Checking a code change or a cheaper AI model	Published results and model decisions
Time	2–30 minutes; recheck on Starter	8–10 hours / 1–2 days (stages 1 / 2); recheck on Starter
Cost	$0.20–1.65	See section 7

3.2 How ReacherX is run

Stage 1 plans one ReacherX cycle on each of the 20 prompts, including companies. For the learning test, each chosen prompt runs twice: once normally, and once with learning switched off. Comparing the two shows what learning adds, separately from everything else that changes over time. As agreed, only one workspace runs at a time, which keeps the system stable and avoids hitting the data providers' limits. The two runs for a prompt go back to back, and their order alternates between prompts, so the time of day favours neither.



3.3 How results are judged

Every person or company returned is checked against each requirement of its prompt, using the result plus up to two web searches. Three AI models from different companies act as judges: two grade every result, and the third decides when they disagree. They never see which tool returned the result. None of them comes from OpenAI, whose model ReacherX uses, because judges tend to favour answers like their own.

Before their grades are used, two people, the project lead and the proposal's author, label a sample blind. The judges must agree with them at least as well as a standard threshold (a kappa score of 0.70; the study this method builds on reports 0.84). A few of the people ReacherX rejects are judged too, to catch good matches it wrongly turns away. Supported tools are compared on the same number of results: each tool's top 5 per prompt, within each group.

3.4 Rules that keep the results honest

- Everything that decides a result (prompts, requirements, answer keys, measures and thresholds) is fixed in writing before the first run.

- Publish every result, including losses and “no clear difference”. Comparisons carry appropriate uncertainty intervals; single observations are descriptive. Claim a lead only when supported by the comparison interval.
- The write-up states that ReacherX runs the benchmark and is one of the tools tested, and names every vendor that gave credits. Credits never affect how a tool is set up or scored.
- Each vendor sees its own results for 3 days before publication and can point out a genuine setup mistake. If one is accepted, that tool is re-run and the write-up says so. No vendor can change scores.
- Personal data never goes into the code repository; only totals and scores do.

4. What will be measured

Measure	What it shows
Match quality	Which tool finds the most right people or companies: the share of results that are verified matches, and how many there are, for each prompt group.
Search quality	For ReacherX: how many verified matches each of its searches produces, cycle by cycle, to show whether its searches get better.
Mistakes	For ReacherX’s scoring step, which qualifies a person at a score of 70 or more: how often it accepts a wrong person, and how often it rejects a right one.
Freshness	How recent the evidence is, and whether roles, employers, funding events and job openings are current. Signal prompts use the fixed reference date.
Speed	Time from submission to the first and fifth results later verified as matches, including required planning, lookups and waits. Request latency is separate; unreached targets are reported.
Cost	Usage cost at recorded non-sponsored rates per verified match. Report plan assumptions and unpriced credit units; missing prices are not zero. Show cash outlay after grants separately.
Profile data	For ReacherX’s matches, how complete and accurate the profile details are. Verified emails and phone numbers are not tested here; a separate benchmark will cover them.
Self-improvement	How many more verified matches ReacherX finds with learning on than with learning off, prompt by prompt.

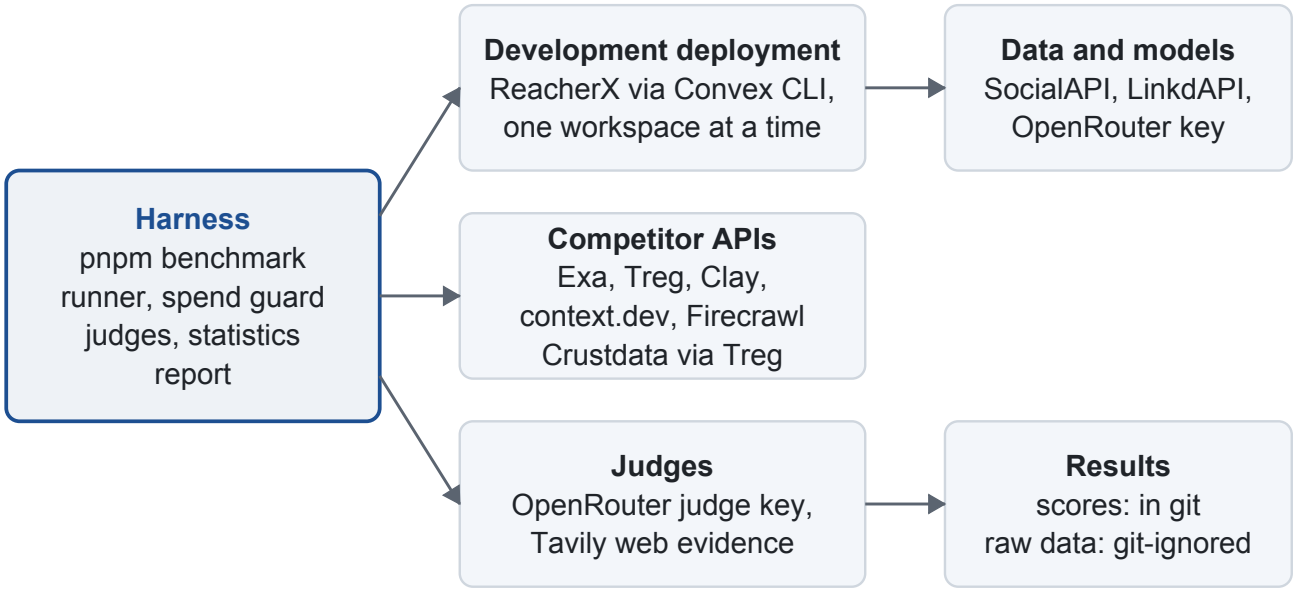
ReacherX searches live sources and qualifies people over cycles. Fetching, qualification, enrichment and scheduled waits can add latency; Starter limits also affect runtime. These conditions are disclosed, with waits included in elapsed time. Freshness is checked from evidence, not assumed from the real-time approach.

5. Technical design

This section is for the engineers and coding agents who build and run the benchmark.

5.1 Architecture

The benchmark is a TypeScript command-line harness plus a few benchmark-only Convex functions. The harness drives the development deployment and the competitor APIs, judges the results and writes the report. It uses tsx, the AI SDK and the OpenRouter provider, all already in the repo. Shared parts sit in benchmarks/core, so later benchmarks, such as the enrichment one, reuse them unchanged.



5.2 Components to build

The new pull request creates this structure; none of it is on main yet. Only the Convex benchmark functions already exist, on PR #139's branch, and are copied from there; PR #139 itself stays untouched.

Path	What it holds
benchmarks/ core/	**runner**: runs one step of the frozen plan, with a checkpoint after every unit of work, so a failed step resumes without paying twice. **convex**: drives the development deployment through the Convex CLI, as round 1 did, passing secrets through standard input so they never appear in a command line. **spend**: reads each key's spend and sets each step's credit limit. **judges**: evidence, the blind panel and the tie-break. **stats**: Wilson intervals, paired bootstrap, Holm correction, Cohen's kappa. **report**: Markdown tables with ASCII charts, in the style of the TOON README; no dashboard.

Path	What it holds
benchmarks/ people-search/	**plan**: prompts, criteria, answer keys and thresholds, frozen before the first run. **tools**: one adapter per tool, all with one interface: a prompt in; typed people or company results, cost, time and the raw response out. Declare supported tasks; preserve prospectType, entity identity/domain and evidence dates. **results**: totals and scores in git; raw responses and people's data in a git-ignored folder.
convex/ benchmarkPeopleSearch.ts	Benchmark-only functions copied from PR #139: create and start workspaces, report run status, export candidates, stop and clean up.
convex/ benchmarkReplay.ts	New, benchmark-only: passes a frozen candidate's stored inputs to ReacherX's own qualification function.

5.3 How a live step runs

1. The runner loads the frozen plan and the step's approved amount, sets the keys' credit limits, and switches the deployment's run settings on.
2. ReacherX, one workspace at a time: create it from the prompt, start its cycles, check its status every 30 seconds, wait for the last qualification and lesson writing, read the deployment key's spend, export the candidates, and pause the workspace.
3. Competitors, one request at a time: each adapter saves the raw response with its cost and time, and for Treg the provider that served the call; results that repeat another tool's are flagged.
4. Judging: evidence for each person or company, the blind panel, and the tie-break where the two judges disagree.
5. Statistics and the report, printed to the console and written to the results folder; at a decision point, the one-page checkpoint report.
6. Reset, which runs even when a step fails: background jobs paused, automatic cycles off, and the original development keys restored.

5.4 Commands

Command	What it does
pnpm benchmark	The quick check (replay) with the current model.
pnpm benchmark --model <id>	The quick check with another model.
pnpm benchmark --live <step>	One live step: smoke, pilot, main or stage2. It stops at the next decision point.
pnpm benchmark --dry-run	The whole pipeline on saved responses, with no live calls, at $0.

Keys are read from a git-ignored environment file and never printed. Output is plain Markdown and JSON, so people and coding agents read it the same way. A new tool needs only an adapter, and a new benchmark only its own plan and adapters.

5.5 Run setup

- **Same as production:** the models in docs/configuration.md (gpt-6-luna-pro for query writing and qualification, gpt-oss-120b for enrichment, and production's lesson-writing and embedding models), search batch sizes, the 90-day X lookback and the provider search time limit. Concurrency is adjusted for Starter, as below.
- **Only during a run:** automatic cycles on and background jobs unpaused (both off on development by default), an initial 10-minute cycle gap, the first-hour bootstrap cycles included, and only the workspace being run active.
- **Why 10 minutes:** round 1 discovery took 7–9 minutes, with qualification taking 5–6.5 more and lessons written afterwards. Production's 2-minute gap is too short to learn from the last cycle. On Starter, start with a 10-minute gap; each cycle waits for qualification and lessons. The smoke test and pilot measure timing. Record and restore non-secret env values and effective pool limits for each run.
- **Learning off:** no learned lessons or similar past cases in query writing and qualification, and no ranking of queries by past results, including the 7-day pause for X queries with no qualified people in three searches. Wider searches over cycles, fetching only newer posts and skipping people already seen stay on in both groups.
- **Learning switch:** a benchmark-only setting naming the learning-off workspaces. It gates lessons, similar past cases and query ranking only there, and is never set in production. It alone changes discovery behaviour; concurrency wiring is development-only.
- **Keys:** two OpenRouter keys funded by the project lead, each with its own limit, one for the development deployment and one for the judges. With one workspace at a time, the deployment key's spend before and after a workspace is its exact OpenRouter cost; embeddings, possibly billed by OpenAI directly, are a fraction of a cent per cycle and are estimated. Production X and LinkedIn keys with balance come from the project lead, for runs only.
- **Starter setup and speed:** tune workflow and worker pools together to Starter limits, identically in both learning groups. Verify in the smoke test; speed results apply to Starter. Record the cycle schedule and separate active processing from scheduled waits.

5.6 Quick check (replay) details

About 300 candidates from the full run are frozen with their labels and everything qualification read: the evidence posts, the profile and the text of linked pages, read once with the benchmark's Exa key (at most 2 per candidate, at most $1.20 in total). The set stays out of git, in the development deployment's file storage. A benchmark-only Convex function passes these inputs to ReacherX's own qualification function, so precision, recall and cost can be compared for a code change, another model, or with and without lessons, while the inputs stay fixed. To try another model, the replay sets the qualification model on the development deployment while it runs, restores it afterwards, and checks the model recorded on every call; nothing else runs meanwhile. It costs $0.003–0.0055 per candidate, and only the AI calls cost money.

5.7 Judging and statistics

- Judges: GLM 5.3 Flash and DeepSeek V4.1 Flash, with Qwen3.8 Flash deciding disagreements. Gemini is not used. Each result is graded on every criterion as met, partly met or not met, with a link to the evidence.
- ReacherX's top 5 are its five best-fit qualified entities of the required type. Exa, Treg, context.dev and Firecrawl are judged on their top 10, and top 10 is reported where judged. Crustdata is judged at top 5. Up to 3 random rejected people per prompt per cycle are judged for the error rate.
- Use 95% Wilson intervals for binary proportions, not cost or time. Aggregate estimates and tool comparisons use paired bootstrap over prompts within each group, with Holm correction across comparisons. Single observations are descriptive. Commit the plan before live runs (pre-registration).

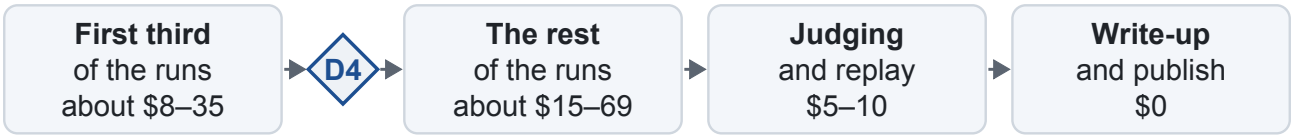
6. Plan

The work runs in two stages with four decision points, D1 to D4. At each one the run pauses, and a one-page report on spending, results and any problems goes to the project lead, who decides to continue, adjust or stop. The prompts and measures never change after results have been seen. Stopping early is a valid outcome, and everything measured up to that point is still published.

Stage 1: the next pull request (estimate $24–43)



Stage 2: the full run (estimate $28–114, sized at D3)



D1–D4: see section 6.4. Each stage has a 3-day vendor preview before publication.

6.1 Proving the benchmark works first

Check	What it shows	Cost
Offline tests	Saved round 1 data and organization fixtures work, with no live calls	$0
Repeat round 1	The new code reproduces round 1's scores exactly	$0
Smoke test	One company prompt runs end to end, checking organization identity and result typing	about $1
Pilot	Two people prompts, four cycles per learning condition: 16 cycles	$6–11
Judge check	Two people label about 40 mixed people/company results; the judges must agree	$0

6.2 Stage 1: compare the tools

After the checks, the remaining 17 prompts run through ReacherX's first cycle. Competitors run all 20 prompts on supported native endpoints, with company results kept separate. Everything is judged and 60 more results are labelled by people. The quick-check set is then built and used to compare the current AI model with a cheaper one. The write-up marks the learning result as preliminary, and is published after the vendor preview.

6.3 Stage 2: test learning

ReacherX runs 7–10 prompts for 5–8 cycles each, with learning on and off: the five people signal-based prompts, plus 2–5 drawn at random from the seven people static-filter prompts before any results are seen. This is 70–160 cycles across both conditions, budgeted as fresh paired workspaces. Set the size from measured costs at D3; any approved reuse is itemized before deducting cycles or costs. Repeat the quick check with and without lessons, then judge, preview and publish the results.

6.4 Decision points

Point	Continue if	Otherwise
D1, after the smoke test	A cycle costs no more than $0.65, its Starter runtime is measured, and everything is recorded correctly	Stop, find the cause, and re-estimate
D2, after the pilot	The judges agree with people (kappa of 0.70 or more), learning has time to happen between cycles, and the remaining stage 1 budget and timing, based on the pilot, are approved	Fix the problem, run a smaller main run, or stop
D3, end of stage 1	Every check has passed, each pilot prompt found matches, and stage 2's budget, based on measured cost, is approved	Resize stage 2, or publish stage 1 on its own
D4, a third into stage 2	Cost per cycle is on plan and nothing has failed	Shrink the rest, or pause and fix

D3 matters most: by then stage 1 has already delivered the tool comparison, trusted judges and the cheaper-model answer, and stage 2 adds one answer, whether ReacherX learns.

7. Budget

There is no fixed budget up front. The base allowance for dollar-priced calls is $52–157, using round 1's $0.32–0.65 per cycle and the documented Crustdata rate. Quota-covered services in section 7.3 are tracked separately. Freeze rates and plan assumptions before comparison; unpriced units are reported, not assigned $0. The pilot sets later budgets. Credits affect cash required, not comparable usage cost; retries or changed rates require a revised estimate and approval.

Cost item	Stage 1	Stage 2
ReacherX runs	34 cycles: $11–23	70–160 cycles: $23–104
Other tools	$5–6	none
Judging and web checks	$6–10	$3–6
Quick checks	$2–4	$2–4
Base allowance	**$24–43**	**$28–114**

7.1 Spending over time

Step	Base allowance	Running total
Build and offline checks	$0	$0
Smoke test, then D1	about $1	about $1
Pilot and judge check, then D2	$6–11	$7–12
Main run: 17 more first cycles and 19 remaining competitor prompts	$10–18	$17–30
Judging the main run	$5–9	$22–39
Quick check: current model against a cheaper one, then D3	$2–4	$24–43
Stage 2, first third, then D4	about $8–35	$32–78
Stage 2, the rest	about $15–69	$47–147
Judging stage 2	$3–6	$50–153
Quick check with and without learned lessons	$2–4	**$52–157**

7.2 What the estimate is based on

- Stage 1: 1 company smoke test + 2 people prompts x 4 cycles x 2 conditions + 17 first cycles = 34. Stage 2: 7 x 5 x 2 to 10 x 8 x 2 = 70–160 cycles; no reuse is assumed.
- Historical cycle cost: $0.32–0.65 ($0.28–0.52 AI + $0.04–0.13 X/LinkedIn). Stage 2 runs therefore cost $22.40–104 before rounding. The pilot measures later learning and enrichment overhead.
- Judging allowance: $0.001–0.003/result; verify company grading in the pilot. Tavily: 1,000 free searches/month, then $0.008/search. Confirm quotas before running.

- Treg search: $0.007/search; Lusha: $0.00499/result; Apollo: $0.026/name lookup; Crustdata: $0.009/result (~$0.90 for 20 prompts at top 5). Optional enrichment is off. Rounded competitor allowance: $5–6.
- Engineering and upgrades are excluded. Extra calls, retries or changed prices require a revised estimate.

Learning uses 7–10 people prompts at 5–8 cycles per condition. Repeat runs and 24-hour runs are excluded. Apollo, Lusha, Clay and Crustdata use top 5 unless measured costs justify 10. These limits reduce cost and power; D1–D4 set subsequent budgets and run size.

7.3 Accounts and funding

Account	Status	Needed
OpenRouter	The benchmark key's account, shared with ReacherX's deployments: $4.22 on 10 November	Two funded keys with their own limits, together within the amount approved for each step
Treg (two teams)	$0.06 and $0.23 on 10 November	A grant or top-up. Allow about $4.15 at top 5 including Crustdata, or $8.30 if those tools all return top 10; about $15 allows reruns. Recheck actual costs at D2
Crustdata (via Treg)	Uses the Treg balance; no separate grant assumed	Included above: about $0.90 for 100 search results; optional enrichment is extra
Exa	$19.80 on 10 November	Nothing; stage 1 needs under $2
Firecrawl	1,004 credits on 10 November	Nothing; stage 1 needs 40 credits
context.dev	50,000 credits granted to the project lead's account	Nothing; about 860 needed, at 43 per call in round 1
Clay	Own API: 9,940 of 10,000 results left on 7 November, resetting 18 November	Nothing; about 100 results
Tavily	495 of 1,000 free searches used on 10 November	About $5 beyond the free searches, in the judging line
LinkedIn data: LinkdAPI, now Renidly (production key)	No balance API. Our records: 1,000 credits added, about 72 used. Renidly has granted 3,000 credits to the project lead	Allow about 310–480 credits in stage 1 and 630–2,240 in stage 2 (9–14 units/cycle allowance). Confirm remaining grant balance on the key's account; live ReacherX also uses it
X data (production key)	No balance API	A key with balance from the project lead; about 10 requests per cycle

Figures are historical (7–10 November). Recheck credits, rates, plans and Clay's quota before live calls; the development LinkedIn key was empty. Record quota usage separately.

8. What this fixes in the earlier plan

The earlier plan was checked against the ReacherX code and the vendors' own pages. These are the gaps found, and how this proposal closes each one.

1. **A rising curve could not prove learning.** Results also change for other reasons: searches widen on a schedule, only newer posts are fetched, and people already seen are skipped. Closed by the learning-off comparison.
2. **Good people wrongly rejected were never counted.** Closed by judging some rejected people, and by the quick check.
3. **A full run cannot check every code change.** Ten cycles take about 3 hours per workspace and cost money, and two runs see different posts. Closed by the quick check.
4. **Agreement between AI judges was treated as proof.** On one leaderboard dataset, a study of 350+ models found 60% agreement when both models were wrong. Closed by the human check.
5. **Freshness, speed and cost had no definitions.** Closed by section 4.
6. **"100% fresher data" had never been measured.** ReacherX reads X and LinkedIn through third-party services (SocialAPI, and LinkdAPI, now named Renidly), and LinkedIn post search is sorted by relevance, not date. Closed by the freshness measure.
7. **Some prompts could not be graded or did not fit.** Closed by the approved prompt groups, fixed criteria, a reference date and separate reporting of the company prompts.
8. **Two competitors were not as described.** ZoomInfo cannot be reached through Treg, and Firecrawl Alexandria is a gateway, not a search engine. Closed by the tool list.
9. **Public test sets were a poor fit.** Exa's set covers business lookups only, and OpenBenchmarks' sets cover company news and coding. The 20 custom prompts are used instead.
10. **Faster cycles on the live product would affect every customer.** Closed by using a private development copy.
11. **Fast cycles are not real hours.** A gap of minutes finds few new posts. This is stated in the results; a 24-hour run at the real pace can be added later.
12. **Costs were not recorded per workspace.** Closed by reading the AI key's spend before and after each workspace.
13. **The scoring model could not be changed on its own.** It shares a setting with query writing. Closed by the quick check, which runs only the scoring step.

Appendix A: The 20 prompts

Approved wording and stable prompt numbers, grouped in the four agreed categories. All time windows use one fixed reference date. Signal-based prompts also probe freshness; where no age limit is stated, report the evidence date without adding a recency requirement.

1. People, static filters (7)

Role, location, industry or company size using standard profile attributes.

1. “Find software engineers based in London with 5+ years of experience.”
2. “I'm hiring a senior backend engineer in Amsterdam. Find people I could reach out to.”
3. “Find product managers working at health-tech startups in the United Kingdom.”
4. “Find data scientists working in banking or finance in New York City.”
5. “Find marketing leaders at B2B SaaS companies in the US with 50-500 employees.”
9. “Find CEOs of software companies headquartered in the United Kingdom.”
10. “Find heads of product at fintech companies headquartered in the US.”

2. People, signal-based (5)

Events or content: posts, job changes, funding and technology mentions.

6. “Find developers in Germany who have posted about React in the last 90 days.”
7. “Find software engineers in the US who joined a new employer in the last 90 days.”
8. “Find founders in India whose companies announced a funding round in the last 6 months.”
11. “Find indie developers who build with Next.js.”
12. “Find marketers who mention using HubSpot.”

3. Company, static filters (5)

Industry, location, size or stage using company attributes.

14. “Find early-stage AI startups in North America.”
15. “Find healthcare companies in the United States with 200-1,000 employees.”
16. “Find marketing agencies in Brazil that offer SEO services.”
17. “Find B2B SaaS companies in Europe with 50-200 employees.”
20. “Find e-commerce brands headquartered in the United Kingdom.”

4. Company, signal-based (3)

Funding announcements or active job postings.

13. “Looking for: European fintech companies, Series A stage, funding announced in the last 12 months.”
18. “Find tech companies in the US with 100-1,000 employees that are currently hiring backend engineers.”
19. “Find climate-tech companies in Europe currently hiring data scientists.”

Grade company identities separately; verify ReacherX’s organization-result path in the smoke test. A later harder-query suite uses the same framework.

References

Original sources were checked on 9–10 November 2026; Crustdata, organization handling and statistics were rechecked for this revision. Code references point to the round 1 branch.

The original brief

- The Discord brief and messages of 8–9 November 2026, the thread “Prompts/descriptions that we can use for”, and the shared AI Mode conversation, share.google/aimode/43xrdQqRODgoQnywO.
- The project lead’s review of this proposal, 10 November 2026 (ReacherX Discord, #general), including the screenshot of the model settings from docs/configuration.md.
- The project lead’s approval of the revised 20 prompts and four categories (ReacherX Discord, #general).
- The project lead’s messages of 10 November 2026 on vendor credits: 50,000 credits from context.dev, 3,000 from Renidly, and Treg’s offer to help and to review the setup.

ReacherX code

- convex/socialapiMonitors.ts:37 (api.socialapi.me) and convex/integrations/linkedin/linkdapiClient.ts:144 (linkdapi.com).
- convex/lib/runtimeConfigHelpers.ts:212–309: reschedule interval 60 minutes, allowed range 1–1,440; bootstrap defaults; batch sizes; X lookback of 90 days.
- convex/lib/targetingSpecCore.ts:268–279 (query stages by cycle); convex/workflows/prospecting.ts:2527–2534 (the cycle count only advances during bootstrap).
- convex/workflows/prospecting.ts:1233 (newest posts first), 1239–1244 (lookback) and 1499–1504 (LinkedIn sorted by relevance); convex/lib/twitterProspectingSearchCore.ts:279–284 (only newer posts).
- convex/lib/qualificationCore.ts:445–452 (lessons and similar cases in the qualification prompt); convex/workflows/qualification.ts:399–480.
- shared/lib/qualificationConstants.ts:5 (QUALIFICATION_THRESHOLD = 70).
- convex/lib/queryPrioritizationCore.ts:73 (7-day pause for cold queries) and 93–120 (proven, learning and cold query slots).
- convex/agents/internal.ts:567–617 and 699–714 (learning context for query writing); convex/lib/memoryEvaluatorCore.ts:5 and 48–56 (when lessons are promoted); convex/lib/memoryCore.ts:517–525; no switch to disable learning in convex/convex.config.ts:16–96.
- convex/agents/internal.ts:725 and 949, convex/workflows/qualification.ts:480 (the “onboarding” model setting); convex/workflows/enrichment.ts:431 (“reasoning”); convex/lib/learningCore.ts:213 (“fast”); docs/configuration.md:284–301.
- convex/validators.ts:1409–1412 (results come only from X and LinkedIn).
- convex/workflows/enrichment.ts:365 (only qualified people are enriched); the file reads no memory.
- convex/schema.ts:1840–1860 and 2995–3009 (usage tables); convex/agents/internal.ts:729–748; convex/evaluator.ts:1303–1313; convex/benchmarkPeopleSearch.ts:367.
- convex/jevEvalActions.ts:74–369 (read-only replay); convex/workflows/qualificationAudit.ts:459–480; convex/jevPrefilterShadow.ts:43 (the Jev pre-filter only observes and is off by default).
- convex/lib/planConstants.ts:15–34 (plan limits); convex/workflows/prospecting.ts:221–257 (what happens when a limit is reached).
- convex/lib/autonomousJobHelpers.ts:24–27; convex/workflows/qualification.ts:970–972.
- package.json: tsx, ai, @openrouter/ai-sdk-provider.

- docs/configuration.md:26, 53 and 358–368 (settings are per deployment; scheduling defaults).
- convex/memory.ts:1578–1648 (lessons and similar past cases gathered for qualification).
- convex/lib/qualificationCore.ts:251–296 (the qualification function takes every input, including linked pages, lessons and the model route); convex/lib/qualificationEvaluationCore.ts:63–108 and convex/lib/researchCore.ts:263–320 (linked pages are read through Exa during qualification, at most 2 per candidate).
- docs/configuration.md:201 and 300 (embedding model and how embeddings are billed).
- benchmarks/people-search/harness/core.py:242–256 (reading an OpenRouter key's spend, used in round 1).
- convex/validators.ts:2564 (individual/organization types); convex/lib/enrichmentCore.ts:156–174 (organization enrichment); convex/lib/prospectIdentityCore.ts:37–43 (company profile identity).

Round 1 data

- Round 1 ReacherX run logs, six single-cycle runs: model cost $0.28–0.52, 669–898 seconds, with progress logged every 30–40 seconds, 88–113 candidates, 9–12 LinkdAPI units; LinkdAPI testing-tier diagnostics. Stored in benchmarks/people-search/data, which git ignores.
- Round 1 spend log (data/spend.jsonl): 510 judgments averaging $0.0021; Treg search $0.007; Apollo search free; Apollo enrich $0.026; Exa $0.007 per search.
- PR #139 and its README, github.com/VecterAI/reacher-x/pull/139; commit 008d7709 (judges no longer see a tool's own verdict).
- Clay search responses report the period quota: 9,940 of 10,000 results left on 7 November 2026, resetting on 18 November (round 1 notes, .benchmark-dev/PLAN.md).
- In round 1, Treg served a people search through Exa's people search; the serving provider was recorded per call (.benchmark-dev/PLAN.md).

Web pages

- exa.ai/blog/exa-3 (Exa 3.0).
- docs.exa.ai/reference/search (the "people" category).
- exa.ai/pricing; Websets: exa.ai/docs/websets/pricing (contents: $1 per 1,000 pages for each content type).
- exa.ai/blog/people-search-benchmark (1B+ people, 50M+ updates per week).
- github.com/exa-labs/benchmarks (people.jsonl, 1,400 queries; graders/people.py).
- treg.to, treg.to/tools/lusha, treg.to/tools/apollo, treg.to/tools/zoominfo (not found), treg.to/people-search.
- zoominfo.com/pricing (sales-led, no public API price).
- docs.lusha.com, prospecting API; lusha.com/pricing.
- docs.apollo.io/reference/people-api-search.
- alexandria.firecrawl.dev; firecrawl.dev/pricing; firecrawl.dev/benchmarks/company-enrichment.
- developers.clay.com/searches; clay.com/pricing.
- context.dev/pricing; context.dev/people-search-api.
- socialapi.me; linkdapi.com and linkdapi.com/pricing; renidly.com (LinkdAPI's new name; the LinkdAPI dashboard now leads there).
- reacherx.com and reacherx.com/pricing ("searches X/Twitter and LinkedIn in real time, around the clock").
- openrouter.ai/api/v1/models (model names and prices on 9 November 2026).
- tavily.com/pricing.
- openbenchmarks.com/web-search; context.dev blog post on OpenBenchmarks.

- convex.dev/pricing; docs.convex.dev/production/state/limits (Starter plan and limits, rechecked for this revision).
- treg.to/tools/crustdata; crustdata.com/apis/people-search (search rates, people/company endpoints and indexed versus real-time modes; checked for this revision).

Research papers and references

- Verga et al. (2024). Replacing Judges with Juries. arXiv:2404.18796.
- Panickssery et al. (2024). LLM Evaluators Recognize and Favor Their Own Generations. arXiv:2404.13076.
- Zheng et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685.
- Goel et al. (2025). Great Models Think Alike and this Undermines AI Oversight. arXiv:2502.04313.
- Kim et al. (2025, ICML). Correlated errors across more than 350 language models. arXiv:2506.07962.
- Vu et al. (2023). FreshLLMs and the FreshQA benchmark. arXiv:2310.03214.
- Shankar et al. (2024). Who Validates the Validators? arXiv:2404.12272.
- NIST/SEMATECH e-Handbook of Statistical Methods, the Wilson interval. itl.nist.gov/div898/handbook/prc/section2/prc241.htm.
- Shi et al. (2026). PeopleSearchBench, with judge agreement of kappa = 0.84 against human annotators. arXiv:2603.27476; github.com/LessieAI/people-search-bench.
- Koehn (2004). Statistical significance tests for machine translation evaluation (paired bootstrap). aclanthology.org/W04-3250.
- Hofman et al. (2023). Pre-registration for Predictive Modeling. arXiv:2311.18807.